

CS-4032: Web Programming

Assignment #02 — 2D Cricket Web Application

Assignment Report

Cricket Blitz — Probability-Based Batting Game

Github: https://github.com/humaa-taj/Web_E_23i0743/tree/main/23i-0743-C-A2

Name: Huma Taj

Roll Number: 23i-0743

Section: C

Date: March 26, 2026

1. Game Logic and Working Mechanism

Cricket Blitz is a single-player 2D cricket batting game built using React and Vite. The game simulates a 2-over (12-ball) innings where the player bats against a probability-based power bar system. The core gameplay loop proceeds through four sequential phases:

Game Phase Sequence:

1. Style Select — The player chooses Aggressive or Defensive batting style.
2. Bowling — A ball animation travels from the bowler to the batsman end over ~1 second using `requestAnimationFrame`.
3. Shooting — The power bar slider activates and bounces left-right continuously. The player clicks 'PLAY SHOT' to freeze the slider.
4. Result — The outcome is determined, scores update, commentary displays, and the game advances to the next ball.

The game tracks runs, wickets, and balls bowled using React `useState` hooks. The game ends when either 12 balls have been bowled or 2 wickets fall, whichever comes first. A Game Over overlay then displays the final scorecard with total runs, overs played, strike rate, and wickets lost.

Game Rules:

- Total Overs: 2 (12 balls total)
- Total Wickets: 2 — game ends on the second wicket
- Each ball requires style selection before bowling begins
- Scoring: 0, 1, 2, 3, 4, or 6 runs — no extras or wides
- Strike rate is calculated as $(\text{runs} / \text{balls faced}) \times 100$

2. Mapping of Probabilities to the Power Bar

The power bar is the core mechanic replacing pure randomness. Each batting style has a fixed probability distribution across 7 outcomes. The bar is visually divided into colored segments proportional to each probability — so a 40% wicket probability occupies exactly 40% of the bar width.

2.1 Aggressive Batting Style

High-risk, high-reward distribution. Wicket probability is 40%, but boundary probability (4s and 6s combined) is 25%.

Outcome	Probability	Bar Segment
Wicket	0.40	0.00 - 0.40
0 Runs	0.10	0.40 - 0.50
1 Run	0.10	0.50 - 0.60
2 Runs	0.10	0.60 - 0.70
3 Runs	0.05	0.70 - 0.75
4 Runs	0.10	0.75 - 0.85
6 Runs	0.15	0.85 - 1.00
Total	1.00	0.00 - 1.00

2.2 Defensive Batting Style

Low-risk, low-reward distribution. Wicket probability is halved to 20%, but boundary probability drops to 10% (4+6 combined). Dot balls are much more frequent at 30%.

Outcome	Probability	Bar Segment
Wicket	0.20	0.00 - 0.20
0 Runs	0.30	0.20 - 0.50
1 Run	0.25	0.50 - 0.75
2 Runs	0.10	0.75 - 0.85
3 Runs	0.05	0.85 - 0.90
4 Runs	0.07	0.90 - 0.97
6 Runs	0.03	0.97 - 1.00
Total	1.00	0.00 - 1.00

2.3 How the Slider Determines the Outcome

The `buildSegments()` function in `gameData.js` computes cumulative start/end positions for each segment. The slider position (a float from 0.0 to 1.0) is compared against these ranges. The `getOutcomeFromPosition()` function uses `Array.find()` to return the matching segment. This makes outcome selection entirely deterministic — no `Math.random()` is involved in the outcome logic.

Example: If the slider stops at 0.32, it falls within the Wicket segment (0.00–0.40), so the result is OUT.
If the slider stops at 0.92, it falls within the 4 Runs segment (0.75–0.85 aggressive / 0.90–0.97 defensive).

3. Implementation of Animations

3.1 Bowling Animation

The ball travels from the bowler's position (x=120, y=270 in the SVG viewBox) to the batsman's position (x=420, y=300) over 1000ms. This is implemented using requestAnimationFrame in the startBowlingAnimation() function. A normalized time value t (0 to 1) is calculated as elapsed/1000, and ballPos state is updated each frame. The SVG ball element's cx and cy attributes are computed as linear interpolations between start and end coordinates.

3.2 Batting Animation

The batsman's bat is an SVG <g> element with a CSS transform applied based on the batAnim state. Three states exist: idle (0 degrees rotation), swing (-60 degrees), and hit (+30 degrees). When PLAY SHOT is clicked, the state transitions from swing to hit after 150ms, then returns to idle after 500ms using setTimeout. The CSS transition property on the SVG group element smoothly interpolates between these rotation states.

3.3 Power Bar Slider Animation

The slider is animated using requestAnimationFrame in the runSlider() callback. It maintains a direction value (+1 or -1) and moves the slider position by speed * deltaTime each frame, where deltaTime is computed from the timestamp parameter provided by the browser. The slider bounces when it hits 0.0 or 1.0, reversing direction. This produces a smooth, continuous back-and-forth motion independent of frame rate.

3.4 Result Flash

When a shot outcome is determined, a resultFlash state object is set with the label (e.g., '+4' or 'WICKET!') and a type flag. This renders a positioned overlay div on the cricket field with CSS transition effects. The flash is cleared after 1800ms using setTimeout, at which point the game transitions to the next ball or the Game Over screen.

3.5 Commentary System (Bonus)

The COMMENTARY object in gameData.js stores 3–4 unique commentary strings for each outcome type (Wicket, 6 Runs, 4 Runs, 3 Runs, 2 Runs, 1 Run, 0 Runs). After each shot, the getCommentary() function randomly selects one string for the matched outcome and displays it in the commentary box below the cricket field.

4. Gameplay Screenshots

All screenshots were captured during live gameplay in the browser. Each image below corresponds to a required screenshot scenario as per the assignment submission guidelines.

Screenshot 1 — Aggressive Batting in Action

The player has selected Aggressive batting style. The power bar is visible showing the Wicket segment occupying 40% of the bar, with the slider moving. The scoreboard shows 12 balls remaining, 2 wickets, at 0.0 overs.

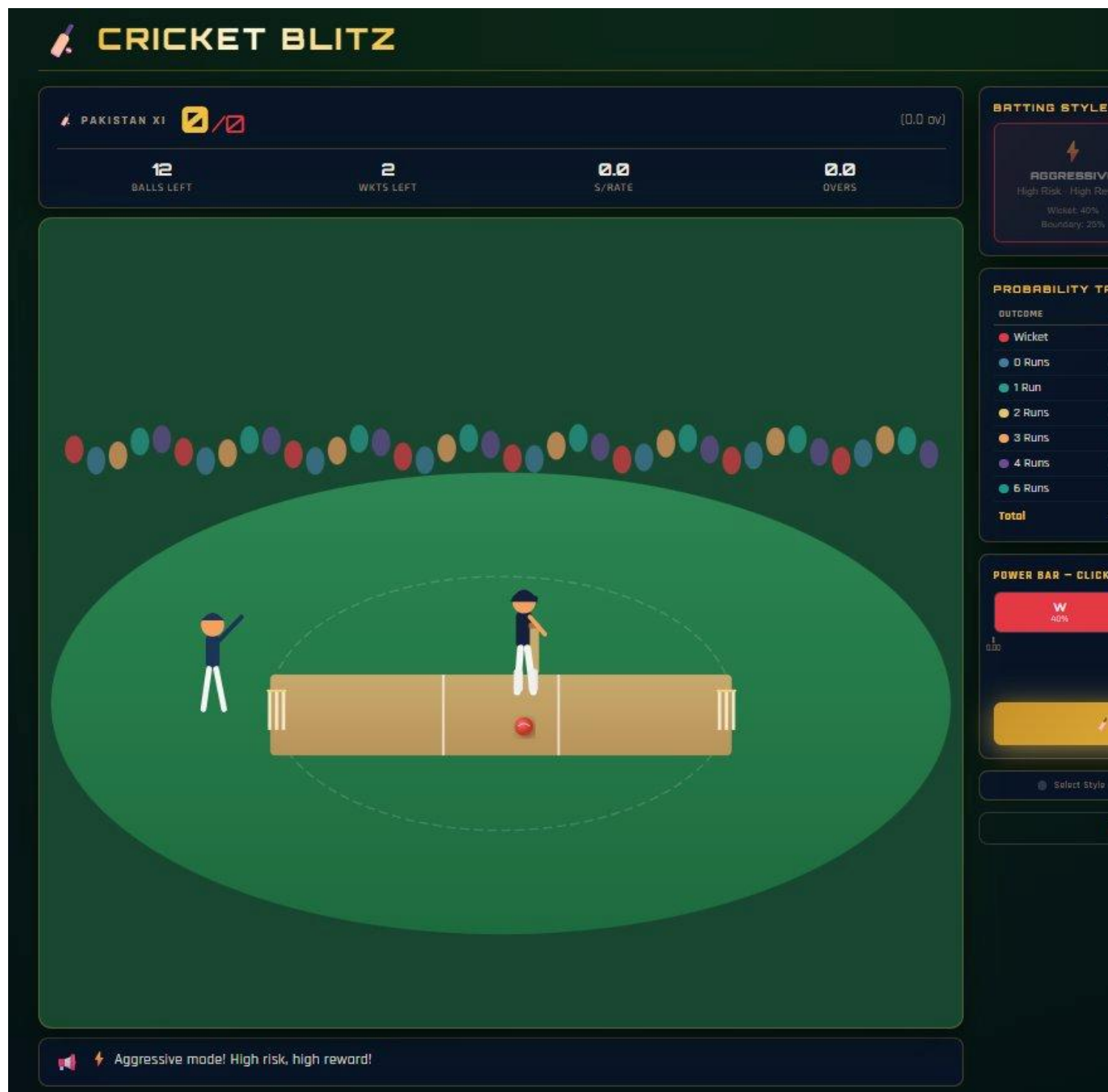


Figure 1: Aggressive batting mode — power bar active with slider, probability table visible (Roll: 23i-0743-C)

Screenshot 2 — Wicket Outcome / After First Dismissal

The player was dismissed — the WICKET! badge is visible on the scoreboard. The commentary reads 'Out! Caught behind! What a stunning dismissal!' Scoreboard shows 11 balls left, 1 wicket remaining, at 0.1 overs.

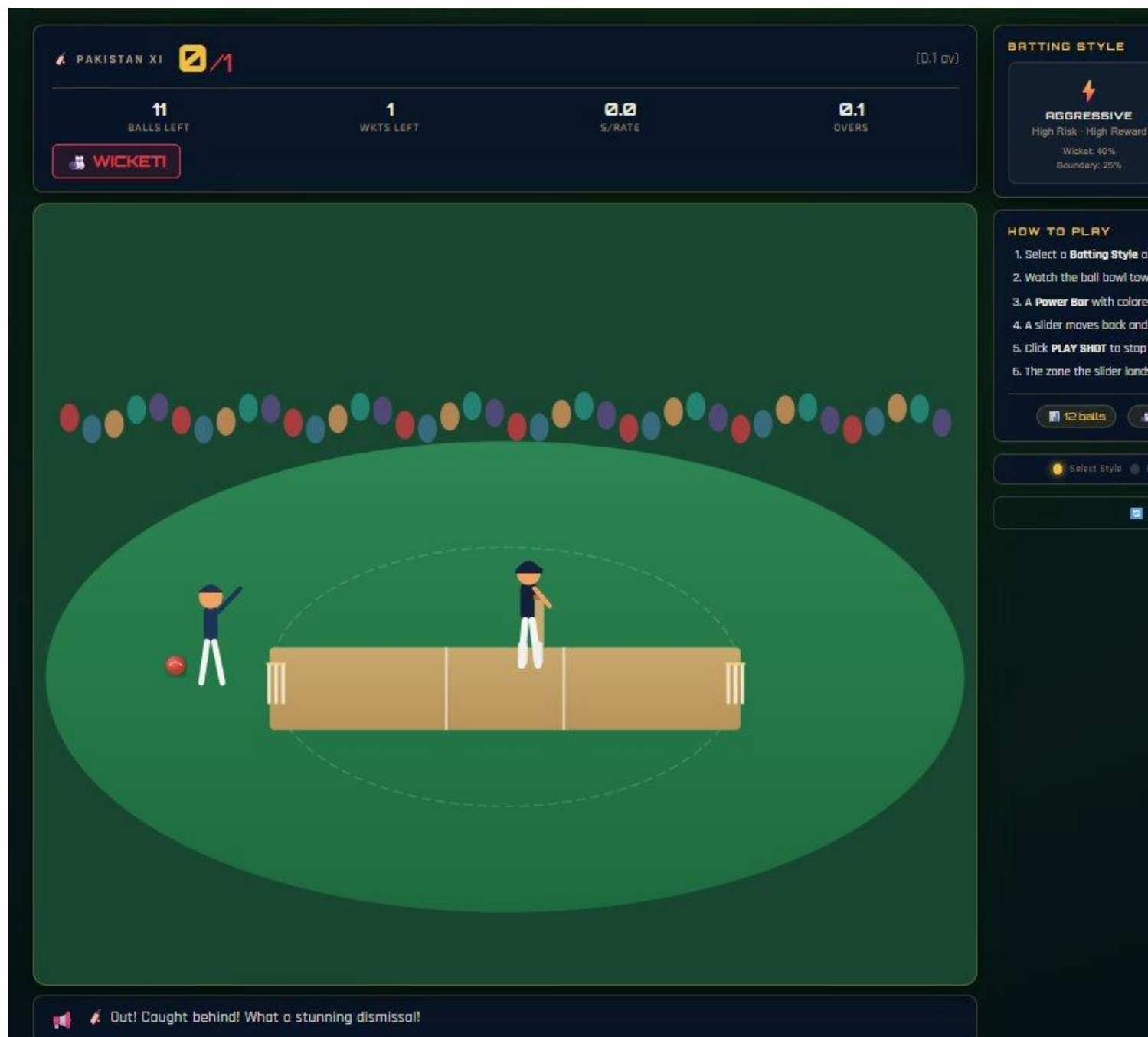


Figure 2: Wicket result — WICKET! banner shown, commentary updated, scoreboard reflects 1 wicket fallen (Roll: 23i-0743-C)

Screenshot 3 — Defensive Batting with +4 Runs Result

The player used Defensive batting style and scored 4 runs. The +4 RUNS badge is visible as an overlay on the field. The probability table on the right shows the Defensive distribution (20% wicket, 30% dot balls). Scoreboard shows 10/1 at 0.5 overs, 200.0 strike rate.

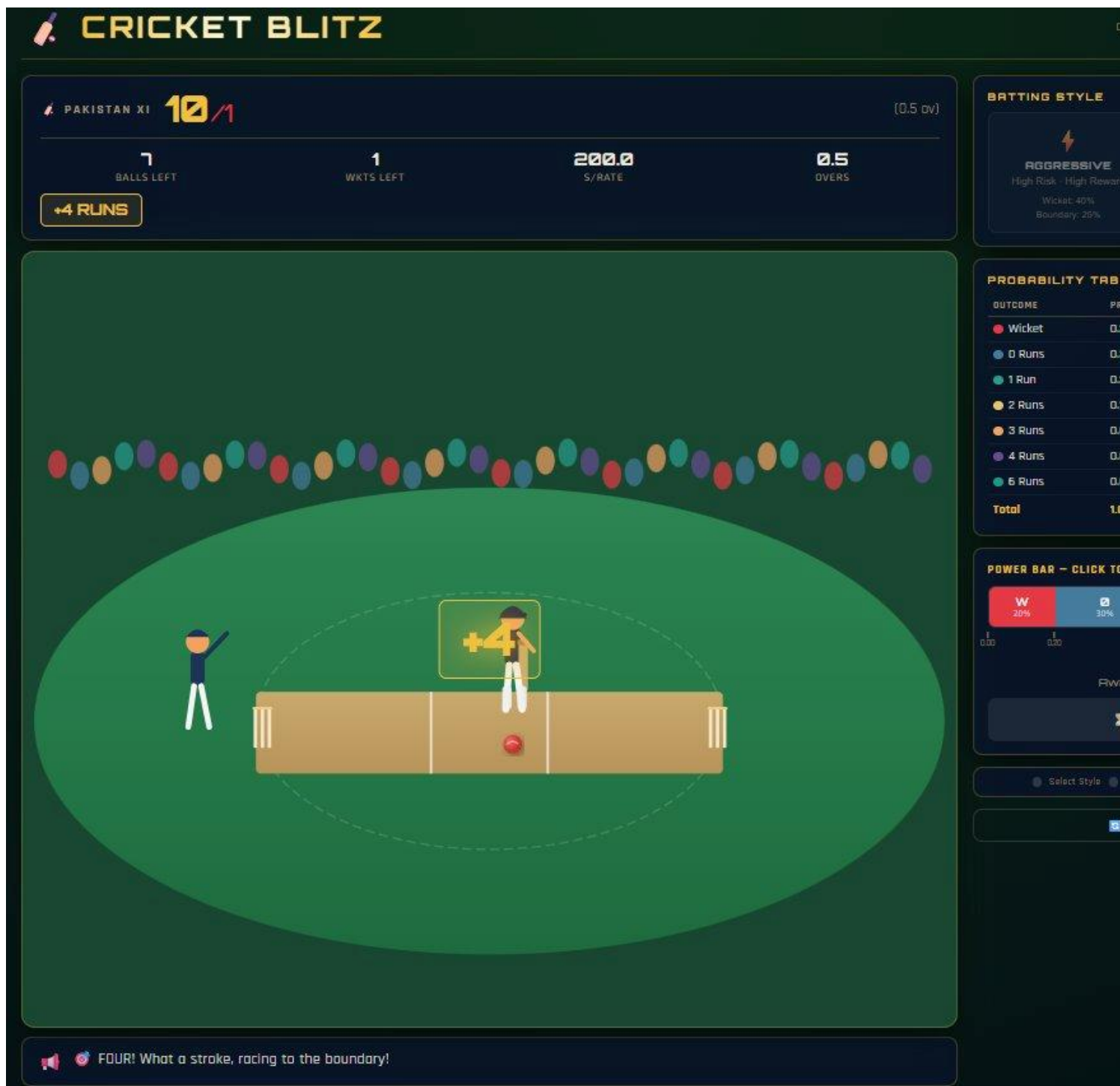


Figure 3: Defensive batting — +4 Runs outcome, result flash overlay visible, defensive probability table shown (Roll: 23i-0743-C)

Screenshot 4 — Game Over Screen

The innings ended after both wickets fell. The 'Innings Complete' overlay shows the final scorecard: Total Score 10/2, Overs Played 1.0, Balls Faced 6, Strike Rate 166.7, Wickets Lost 2.

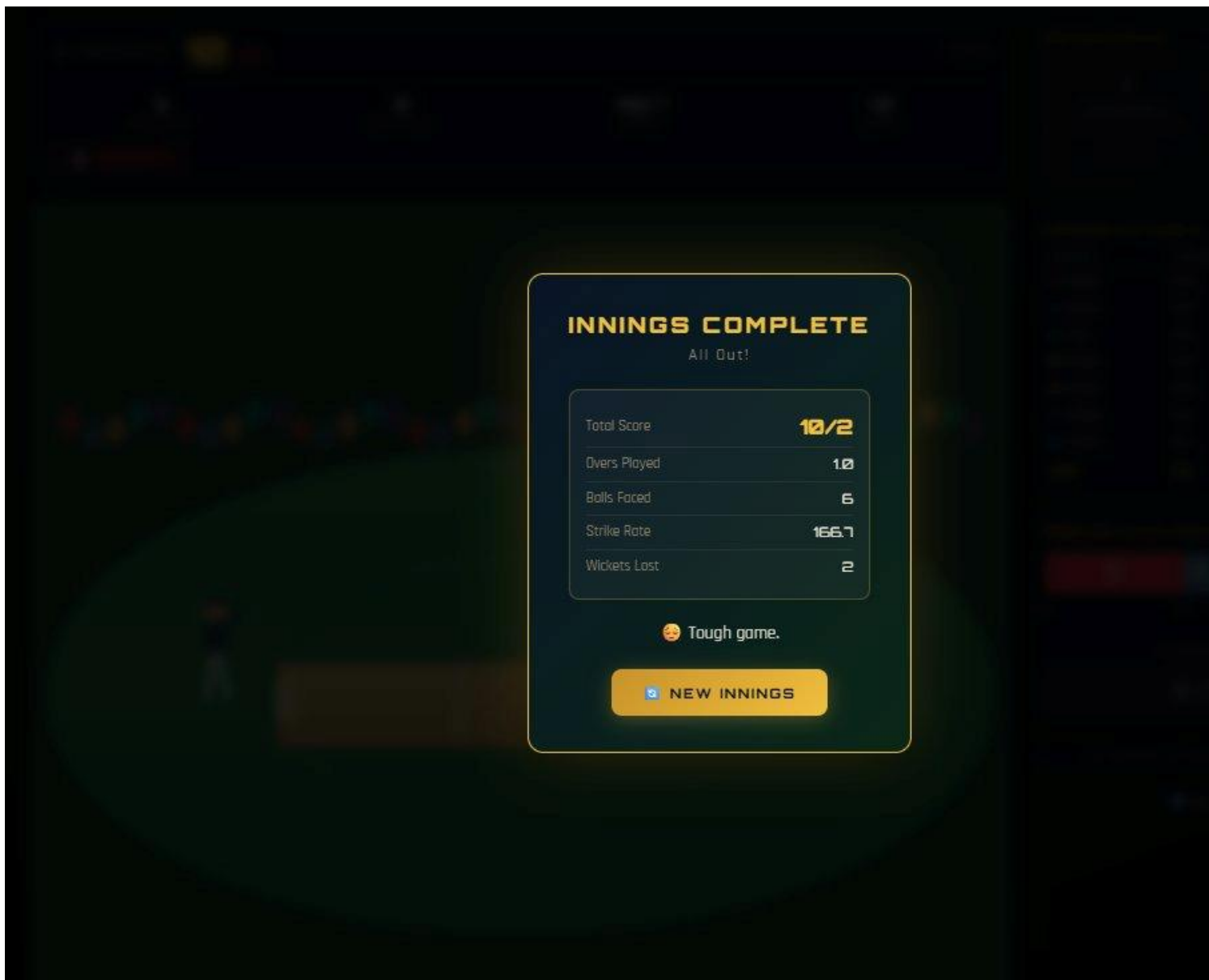


Figure 4: Game Over screen — 'Innings Complete' overlay with full scorecard summary (Roll: 23i-0743-C)

Screenshot 5 — Power Bar Close-Up (Defensive Mode)

A close-up view of the power bar during Defensive mode. Each colored segment is labeled with its outcome and percentage. The slider is positioned at 0.530 (inside the 1 Run zone, 0.50–0.75). The scale below the bar shows the exact probability boundaries.

BATTING STYLE



AGGRESSIVE

High Risk - High Reward

Wicket: 40%

Boundary: 25%



DEFENSIVE

Low Risk - Low Reward

Wicket: 20%

Boundary: 10%

PROBABILITY TABLE - DEFENSIVE

OUTCOME	PROBABILITY	BAR %
 Wicket	0.20	20%
 0 Runs	0.30	30%
 1 Run	0.25	25%
 2 Runs	0.10	10%
 3 Runs	0.05	5%
 4 Runs	0.07	7%
 6 Runs	0.03	3%
Total	1.00	100%

POWER BAR - CLICK TO PLAY SHOT!



Slider: 0.530



PLAY SHOT

☐ Select Style ☐ Bowling ☒ Play Shot ☐ Result

 Restart Game

Figure 5: Power bar close-up — slider at 0.530, segments clearly colored, scale visible below (Roll: 23i-0743-C)

Screenshot 6 — Scoreboard at Match Start

The scoreboard at the beginning of the innings showing the initial state: 12 balls left, 2 wickets remaining, 0.0 overs, 0.0 strike rate. Pakistan XI is the team name displayed.

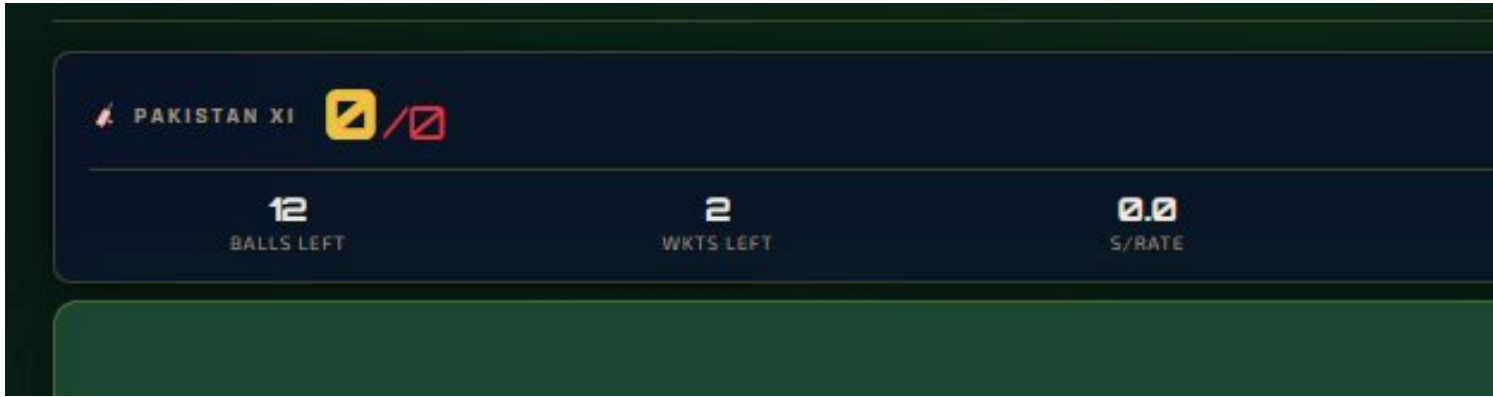


Figure 6: Initial scoreboard — 12 balls, 2 wickets, 0.0 overs at match start (Roll: 23i-0743-C)

5. Technology Stack

Technology	Usage
React 19 + Vite	Component architecture, state management with hooks, fast dev server
JavaScript (ES6+)	Game logic, animation loops, probability computation
SVG (inline)	2D cricket field rendering — field, pitch, stumps, batsman, bowler, ball
CSS (App.css)	Layout, animations, power bar styling, responsive design
requestAnimationFrame	Ball bowling animation and slider movement, frame-rate-independent
React Hooks	useState for game state, useEffect for animation lifecycle, useRef for RAF handles, useCallback for memoization